European Journal of Operational Research 332 (2026) 376–390

Contents lists available at ScienceDirect

European Journal of Operational Research

journal homepage: www.elsevier.com/locate/ejor

Discrete Optimization

Two-phase optimal and heuristic algorithms for flow shop scheduling with reworks under overlapped queue time limits

Hyeon-Il Kim , Dong-Ho Lee *

Department of Industrial Engineering, Hanyang University, Seoul, South Korea

ARTICLE INFO

Keywords:
Scheduling
Flow shops
Overlapped queue time limits
Reworks
Optimal and heuristic algorithms

ABSTRACT

This study addresses flow shop scheduling in which each job is reworked when one of its overlapped queue time limits is violated. The problem is to determine the start times of jobs at each stage and rework setups/operations if occur. After decomposing the problem into the job process route selection and the resulting flow shop or reentrant flow shop scheduling sub-problems, a two-phase optimal branch and bound (TP-B&B) algorithm is proposed for the three-stage makespan problem. The algorithm consists of generating the job process routes using a two-level tree, and finding the resulting optimal schedules using a B&B algorithm while reducing the search space by a dominance property and the best upper bound obtained by solving the resulting scheduling problems. Moreover, for the multi-stage makespan problem, a basic two-phase variable neighborhood search (TP-VNS) algorithm is proposed that solves the two sub-problems iteratively, where each sub-problem is solved using a shaking and local search improvement method with different neighborhood structures. Then, it is extended to a general TP-VNS algorithm with a variable neighborhood descent method. Computational results show that the TP-B&B algorithm gave 21.5% more optimal solutions than the Gurobi while requiring much less computation times for the small sized test instances. Also, the TP-VNS algorithms outperform the existing one significantly. Specifically, the basic (general) TP-VNS algorithm gave 0.7% (1.2%) improvement in the overall optimality gap for small sized test instances, and 1.2% (2.9%) improvement in the overall relative performance ratio for medium-to-large sized test instances.

1. Introduction

Flow shop scheduling is a well-known combinatorial optimization problem that determines the start times of jobs at each serial stage. Due to theoretical challenges and practical applications, various flow shop scheduling problems have been studied since the two-stage makespan problem was solved in polynomial time by Johnson (1954). In the theoretical aspect, most of the problem are NP-hard, and hence various exact and heuristic algorithms, such as branch and bound, meta-heuristics and machine-learning algorithms, are developed. Also, they have a number of practical applications in high-volume and low-variety manufacturing and service systems. Accordingly, the ordinary problem has been extended to the ones with practical considerations such as job arrival times, sequence-dependent setups, no-wait or queue time limits, energy consumptions, and so on. For literature reviews, refer to González-Neira et al. (2017), Rossit et al. (2018, 2021), Komaki et al. (2019), Panwalkar and Koulamas (2020), İnce et al. (2024), Mraihi et al. (2024), Fernandez-Viagas (2025), Ostermeier and Deuse (2024), Perez-Gonzalez and Framinan (2024), and de Athayde Prata et al. (2025).

Among the extensions, this study focuses on the problem with queue time limits. Unlike the no-wait flow shop scheduling problems in which the job waiting times between stages are not permitted, the problem considered in this study allows the limited waiting times. The representative example can be found in wafer fabrications, where the surface of a wafer must be kept clean while waiting for the next process because the over-exposure of wafer surfaces to air for a certain time causes quality problems by possible impurity contaminations. Moreover, a chemical treatment applied at the previous process is only effective for a certain amount of time, and hence the wafer lot must be started on the next process within the time limit. In this case, a wafer lot violating a queue time limit may be scrapped or reworked depending on the wafer features. See Joo and Kim (2009), Joo et al. (2009), Klemmt and Mönch (2012), Jung et al. (2014), Wang et al. (2015), Pirovano et al. (2020), Han and Lee (2023a, b), Wang et al. (2025), and Park and Huh (2026) for more details on queue time limits of wafer fabrication processes.

* Corresponding author.
E-mail addresses: iehikim@hanyang.ac.kr (H.-I. Kim), leman@hanyang.ac.kr (D.-H. Lee).

https://doi.org/10.1016/j.ejor.2025.12.048
Received 16 May 2025; Accepted 31 December 2025
Available online 1 January 2026

Moreover, queue time limits can be found in other manufacturing systems such as TFT-LCD (Hong et al., 2018), steel (Harjunkoski & Grossmann, 2001), furnace tube (Su, 2003), food (Akkerman et al., 2007), bio (Gicquel et al., 2012), battery (Zhou et al., 2018), and automobiles (Yılmaz & Yılmaz, 2022; Yılmaz et al., 2024).

According to Kim and Lee (2023), the queue time limits in flow shops can be classified into four types: (a) single consecutive; (b) single continuous; (c) multiple non-overlapped; and (d) multiple overlapped. Fig. 1 illustrates the four types of queue time limits in the four-stage flow shop. In the figure, $QTL_{m,m'}$ denotes a queue time limit between stages m and m'. As can be seen in the figure, the single consecutive type is the simplest one with a single limit between two consecutive stages, e.g. $QTL_{1,2}$ in Fig. 1(a), and the single continuous type is a single limit between two arbitrary, not consecutive, stages, e.g. $QTL_{1,3}$ in Fig. 1(b). Also, there are two types of multiple queue time limits, i.e. non-overlapped type with two or more consecutive and/or continuous limits without overlapped each other, e.g. $QTL_{1,2}$ and $QTL_{2,4}$ in Fig. 1(c), and overlapped type, e.g. $QTL_{1,3}$ and $QTL_{2,4}$ in Fig. 1(d). Note that the overlapped type is the most general one that can be found in various manufacturing systems, e.g. one between a cleaning process and the first deposition process and the other between the cleaning and the second deposition processes in display manufacturing systems.

This study addresses the flow shop scheduling problem in which each job is reworked after a setup is done on a rework setup station when one of its overlapped queue time limits is violated. The problem is to determine the start times of jobs at each stage and rework setups and operations, if occur, with the objective is to minimize the makespan. For the problem, the following research questions (RQs) are addressed in this study.

RQ1: Is it possible to generate all possible schedules of the problem?

All possible schedules can be generated by decomposing the problem into the job process route selection and the resulting flow shop or reentrant flow shop scheduling sub-problems according to possible reworks depending on violating a queue time limit. The job process route selection sub-problem is to select one of possible process routes for each job among the normal one with the ordinary operations and the abnormal ones with additional rework setup and operations. Also, the resulting scheduling sub-problem becomes the ordinary flow shop scheduling problem if all jobs are processed by the normal process route, and the reentrant flow shop scheduling problem, otherwise.

RQ2: Is it possible to find a more efficient optimal algorithm than the Gurobi?

A two-phase branch and bound algorithm is developed for the three-stage problem. To the best of the authors' knowledge, there is no previous study that develops an optimal algorithm for flow shop scheduling with reworks under overlapped queue time limits. In the algorithm, all possible sets of job process routes are generated using a two-level tree, and for each set of job process routes, the resulting optimal schedule is obtained using a branch and bound algorithm with an active schedule based branching scheme, an NEH based upper bound, and a modified machine based lower bound. Note that the algorithm utilizes a dominance property that can reduce the search space while updating the best upper bound obtained so far by solving the resulting scheduling subproblems, and an idle time insertion method to convert an infeasible schedule that violates a queue time limit for a job with the normal process route to a feasible one. The two-phase optimal algorithm was compared with Gurobi for small sized test instances, and the test results are reported.

RQ3: Is it possible to find more efficient heuristic algorithms than the existing algorithm?

A two-phase variable neighborhood search algorithm is developed that solves each sub-problem iteratively using a shaking and local search improvement method with different neighborhood structures. Then, it is extended to a general variable neighborhood search algorithm with a variable neighborhood descent method that enlarges the search spaces by considering the orders of the neighborhood structures in the shaking and local search improvement steps while generating multiple solutions for each of the neighborhood structures in the local search improvement step. The two algorithms were compared with the existing algorithm, and the results are reported. Moreover, sensitivity analyses were done to examine the performances of the algorithms according to different levels of rework setup times and queue time limits as well as the parameter changes.

The rest of this paper is organized as follows. Section 2 reviews the previous studies on the flow shop scheduling problems with queue time limits, and claims the contributions of this study. Section 3 describes the problem in more details with a mixed integer programming model. Section 4 explains the two-phase branch and bound algorithm for the three-stage problem, and reports the test results, findings and discussion. Section 5 explains the two-phase variable neighborhood search algorithms for the multi-stage problem, and reports the test results, findings and discussions. Finally, Section 6 summarizes this paper and proposes further research topics.

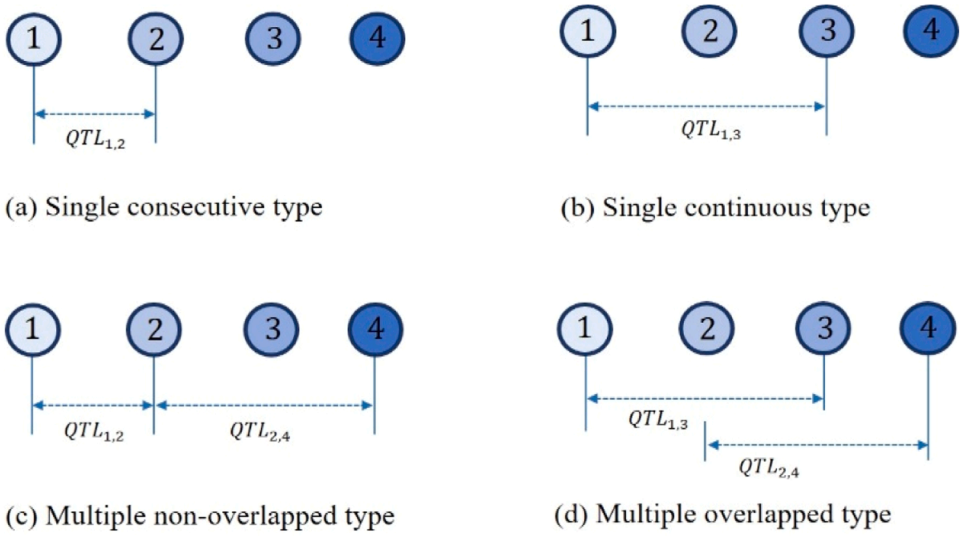


Fig. 1. Four types of queue time limits: examples (adopted from Kim and Lee (2023)).

2. Literature review

This section reviews the previous studies on flow shop scheduling problems with queue time limits, and claims the contributions of this study. As can be seen in Table 1, the previous studies can be classified by the types of queue time limits (single consecutive, multiple non-overlapped, and multiple overlapped), the number of stages (two and multiple), the solution approaches (optimal algorithms, constructive heuristics, and meta-heuristics), the job treatments when violating a queue time limit (scrap and reworks), and objectives (makespan based, flow time based, and due-date based measures).

As can be seen in the table, most previous studies were done on two-stage flow shops with a single consecutive queue time limit. As an earlier study, Yang and Chern (1995) propose an optimal branch and bound algorithm for the two-stage makespan problem after proving its NP hardness, and Joo and Kim (2009) improve it by developing new dominance properties and lower bounds. Later, An et al. (2016) propose another branch and bound algorithm for the problem with sequence-dependent setups. Unlike the above optimal approaches, various heuristic approaches are proposed due to the complexity of the problem. For examples, Ham et al. (2011) propose an integer programming based dispatching approach for the makespan problem with batch jobs, and Li and Dai (2020) propose a hybrid membrane computing meta-heuristic for the makespan problem for a batch machine followed by a discrete machine, where the jobs within each incompatible job family must be processed on the second discrete machine within a limited queue time. In addition, Yu et al. (2017) propose an approximation algorithm that minimizes the total waiting time variation for the problem with operation skips at the first stage, and Lee (2020) proposes a genetic algorithm for the total tardiness problem with sequence-dependent setups at the second stage. For other studies on the two-stage problems, refer to Su (2003), Wang et al. (2013), Han and Lee (2021) and Jeong et al. (2021).

Due to the limited applications, the above algorithms are extended to solve the multi-stage problems. For the makespan problem with a single consecutive queue time limit, Chen and Yang (2006) propose two mixed integer programming models, and report the better one while requiring less computation times. In addition, Joo et al. (2009) propose a priority rule based scheduling policy that minimizes the total tardiness for a multi-stage wafer fabrication process with a single consecutive queue time limit of each job, and Ham (2012) reports a case study of applying the dispatching approach of Ham et al. (2011) to the jobs with a single queue time limit at an etching station in a wafer fabrication process.

Compared with the above studies, not many studies are done on the problems with multiple queue time limits. For the case of non-overlapped type, Jung et al. (2014) propose a rolling horizon based decomposition approach that minimizes the weighted sum of the number of waiting time violations and the preference-weighted job flow times for diffusion processes in wafer fabrication facilities. Han and Lee (2023a) propose a genetic algorithm with a local search method for the three-stage makespan problem with missing operations, and later, Han and Lee (2023b) propose a list scheduling priority rule, a constructive heuristic and two meta-heuristics for the three-stage total tardiness problem with operation skips. In addition, Mansouri et al. (2023) propose a particle swarm optimization algorithm for the multi-stage makespan problem with job release times.

The problems with overlapped queue time limits were studied recently. In fact, Kim and Lee (2019) report that 20% of all process steps are controlled by overlapped queue time limits in semiconductor manufacturing. Note that the non-overlapped type algorithms may give poor solutions or even infeasible solutions for the problems with overlapped queue time limits. As a beginning study, Kim and Lee (2019) propose a branch and bound algorithm that gives the optimal permutation schedules for the three-stage makespan problem with overlapped queue time limit constraints. Later, it is extended by Lee et al. (2021) that propose a genetic algorithm for the multi-stage problem, and Lee (2023) that proposes another genetic algorithm for the three-stage problem with sequence-dependent setups. Also, Wang et al. (2025) propose a schedule recovery method that minimizes the total delay time for semiconductor packaging lines with machine failures and overlapped queue time limits. Unlike these, Kim and Lee (2023) propose variable neighborhood search algorithms for the multi-stage makespan problem with reworks under overlapped queue time limits, i.e. each job is reworked if a queue time limit is violated.

Although the problem considered in this study is the same as that of Kim and Lee (2023), this study has theoretical, methodological, and managerial contributions. First, in the theoretical aspect, an optimal algorithm is developed for the three-stage problem after deriving a

Table 1
Literatures on flow shop scheduling with queue time limits.

	Articles	Two-stage	Multi-stage	Heuristics		Optimal	Scrap	Reworks	Objectives		
				C[1]	M[2]				MS[3]	FT[4]	DD[5]
Single consecutive queue time limit	Yang and Chern (1995), Joo and Kim (2009), An et al. (2016)	√				√	√		√		
	Su (2003), Ham et al. (2011)	√		√			√		√		
	Wang et al. (2013), Li and Dai (2020)	√			√		√		√		
	Yu et al. (2017)	√		√			√			√	
	Lee (2020)	√			√		√				√
	Jeong et al. (2021)	√			√		√		√		√
	Han and Lee (2021)	√		√	√		√		√		
	Chen and Yang (2006)		√			√	√		√		
	Ham (2012)		√	√			√		√		
	Joo et al. (2009)		√	√			√				√
Multiple non-overlapped queue time limits	Jung et al. (2014)		√			√	√			√	
	Han and Lee (2023a), Mansouri et al. (2023)		√		√		√		√		
	Han and Lee (2023b)		√	√	√		√				√
Multiple overlapped queue time limits	Kim and Lee (2019)		√			√	√		√		
	Wang et al. (2025)		√	√			√				√
	Lee et al. (2021), Lee (2023)		√		√		√		√		
	Kim and Lee (2023)		√		√			√	√		
	This study		√		√	√		√	√		

[1] Constructive heuristics.
[2] Meta-heuristics.
[3] Makespan based measures.
[4] Flow time based measures.
[5] Due-date based measures.

dominance property that can reduce the search space and a feasibility condition that can convert an infeasible schedule that violates a queue time limit for a job with the normal process route to a feasible one by inserting the required idle times. Note that the problem with reworks has a much larger solution space than that of Kim and Lee (2019) because a schedule violating a queue time limit is feasible, and hence additional scheduling decisions must be made for rework setups and operations. Moreover, this study is the first one that develops an optimal algorithm for flow shop scheduling with reworks under overlapped queue time limits. Second, in the methodological aspect, the previous approach of Kim and Lee (2023) is improved using the two-phase approach proposed in this study. Note that the previous approach changes only the job sequence at the first stage and determines the resulting schedule at the remaining stages using priority rules. On the other hand, the two-phase approach solves the job process route selection and the resulting scheduling sub-problems iteratively, and hence can search much larger solution spaces than the previous one. Due to the enlarged search space, it is shown that the two-phase branch and bound algorithm gives more optimal solutions than the Gurobi while requiring much less computation times, and the two-phase variable neighborhood search algorithms outperform the existing algorithm significantly. Finally, in the managerial aspect, the solution algorithms proposed in this study can be used as a core advanced planning and scheduling (APS) methodology for a variety of manufacturing systems with reworks under overlapped queue time limits. In fact, this study was motivated from a scheduling expert who developed an APS system of a display manufacturing company in Korea.

3. Problem description

The system considered in this study consists of multiple serial stages and one rework setup station, where each stage is equipped with a single machine with a sufficient buffer capacity. There are n jobs to be processed by multiple operations with a linear precedence relation. However, unlike the ordinary flow shop, each job has two or more overlapped queue time limits, each of which may be a consecutive or continuous type. If a job violates one of its queue time limits at a stage, it must be reworked from the stage where the violated queue time limit starts after a rework setup is done at the rework setup station. Fig. 2 shows an example of M-stage flow shop with two overlapped queue time limits, i.e. one between stages m and m', i.e. $QTL_{m,m'}$, and the other between $m+1$ and M, i.e. $QTL_{m+1,M}$. If a job violates the queue time limit $QTL_{m,m'}$ at stage m^* ($m \le m^* \le m'$), the job returns to stage m, and the resulting rework operations are done from stage m to m^*-1 after the rework setup is done at the rework setup station, where some rework operations can be skipped depending on job features. Otherwise, it moves to the next stage m^*+1. Note that the queue time at stage s for a job with $QTL_{a,b}$ is defined as the sum of waiting times at stages $a, a+1, \ldots, s$, where $s \le b$.

For the system described above, the problem is to determine the start times of the jobs at each stage and the rework setups/operations if occur with the objective of minimizing the makespan. As an early study, a static and deterministic version of the problem is considered in this study, i.e. all jobs are ready for processing at time zero and all data are deterministic and given in advance. Other assumptions for the problem are: (a) one job processed at a time on each stage; (b) sequence-independent rework setup times; (c) negligible transportation times; (d) same processing time of an ordinary and the corresponding rework operations; and (e) rework allowed once for each job due to quality problems. Note that assumptions (a), (b) and (c) are the same as those of the ordinary flow shop scheduling problem, and (d) and (e) were made from the option of the scheduling expert who motivated this study.

According to the possible reworks depending on violating a queue time limit, the problem can be decomposed into two sub-problems: (a) job process route selection; and (b) resulting flow shop or reentrant flow shop scheduling. The job process route selection sub-problem is to select the process route of each job among the normal one with the ordinary operations and the abnormal ones with additional rework setup and operations. Note that each job has two or more abnormal process routes due to the overlapped queue time limits, where each queue time limit can be violated at any of the corresponding in-between stages. Fig. 3 shows all possible process routes of a job with two overlapped queue time limits $QTL_{1,2}$ and $QTL_{1,3}$ in the three-stage flow shop. As can be seen from the figure, the normal process route is $1-2-3$ with ordinary operations, and the abnormal process routes are $1-RS-R1-2-3$ if $QTL_{1,2}$ is violated at stage 2 and $1-2-RS-R1-R2-3$ if $QTL_{1,3}$ is violated at stage 3, where RS denotes the rework setup operation and $R1$ ($R2$) denote rework operation at stage 1 (2). Once the process route of each job is selected, the resulting scheduling problem becomes the ordinary flow shop problem if all jobs is processed by the normal process route, and the reentrant flow shop scheduling problem, otherwise.

Now, the problem can be formulated as the following mixed integer programming model of Kim and Lee (2023) using the notations given in Table 2.

[P] Minimize C_{max}
subject to

$$\sum_{p \in P} Z_{ip} = 1 \qquad \forall\, i \in I \tag{1}$$

$$X_{ipj} = Z_{ip} \qquad \forall\, i \in I, p \in P \text{ and } j \in O_{ip} \tag{2}$$

$$S_{ipj} + C_{ipj} \le M \cdot X_{ipj} \qquad \forall\, i \in I, p \in P \text{ and } j \in O_{ip} \tag{3}$$

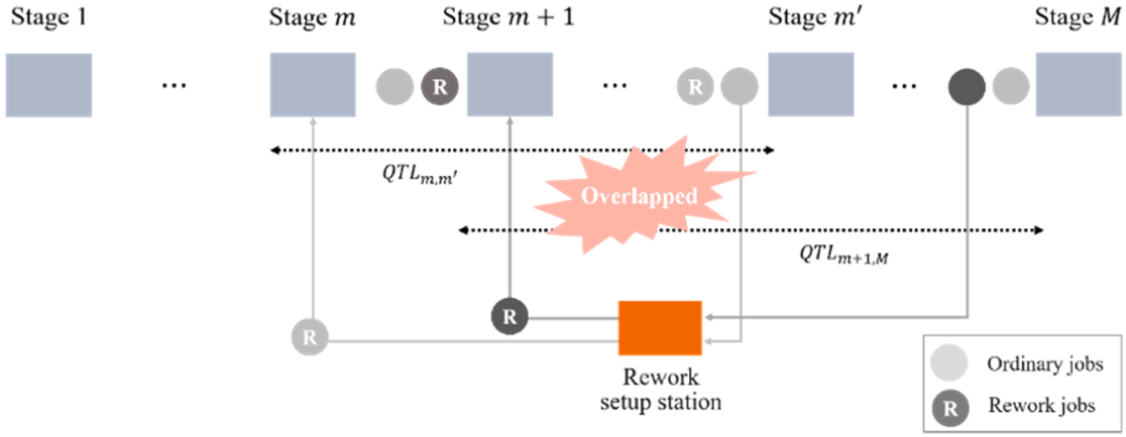


Fig. 2. Overlapped queue time limits in M-stage flow shop: example.

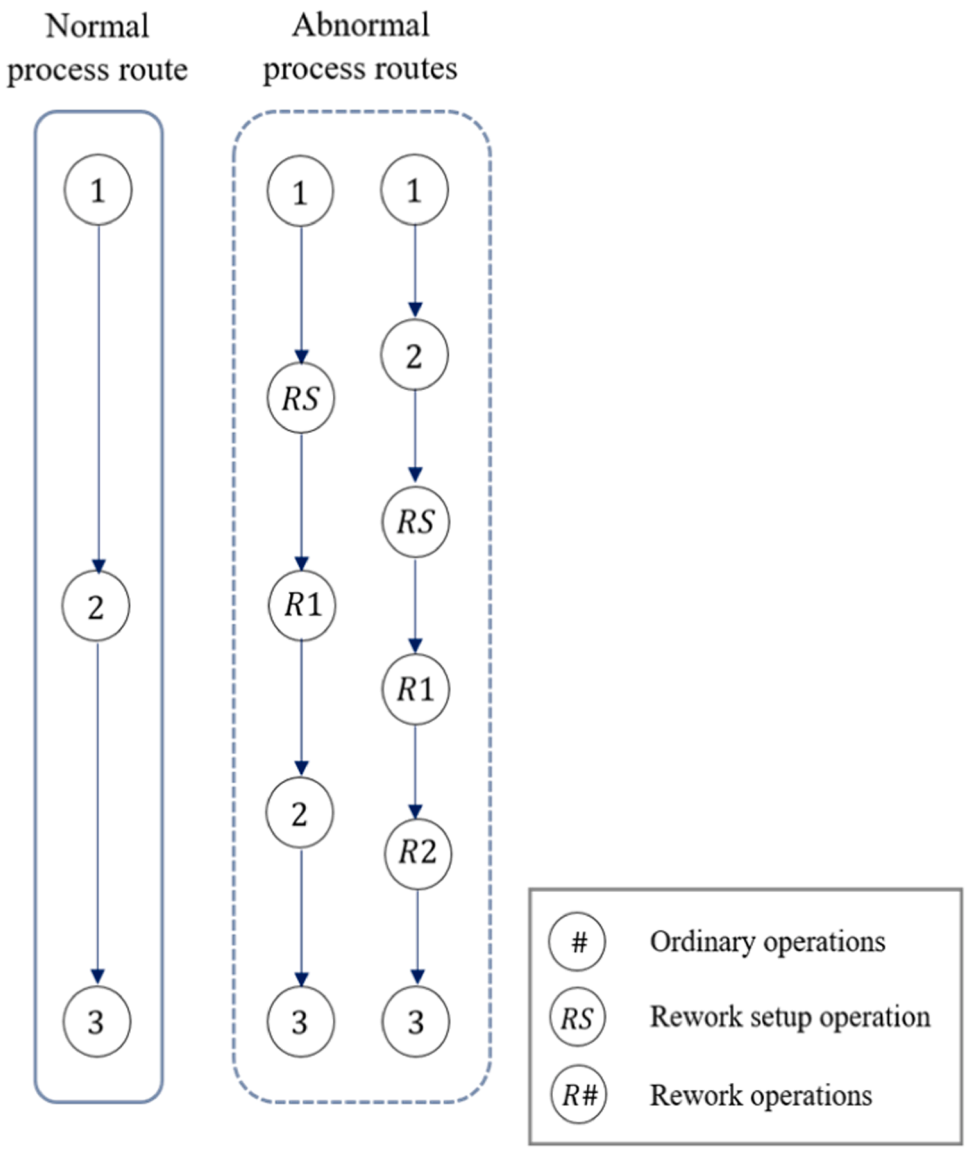


Fig. 3. An example for normal and abnormal process routes.

$$S_{ipj} + t_{ij} \le C_{ipj} + M\cdot(1 - X_{ipj}) \quad \forall\, i \in I, p \in P \text{ and } j \in O_{ip} \tag{4}$$

$$S_{ipj} + M\cdot Y_{ipji'p'j'} \ge C_{i'p'j'} \quad \forall\, i,i' \in I, p,p' \in P, j \in O_{ip} \text{ and } j' \in O_{i'p'}\ (i \ne i') \tag{5}$$

$$S_{i'p'j'} + M\cdot(1 - Y_{ipji'p'j'}) \ge C_{ipj} \quad \forall\, i,i' \in I,\ p,p' \in P,\ j \in O_{ip} \text{ and } j' \in O_{i'p'}\ (i \ne i') \tag{6}$$

$$S_{ipj} \ge C_{i,p,j-1} \quad \forall\, i \in I, p \in P \text{ and } j \in O_{ip} \setminus \{f_{ip}\} \tag{7}$$

$$S_{i1k'} - C_{i1k} \le Q_{ikk'} \quad \forall\, i \in I \text{ and } k,k' \in O_{ip}\ (k < k') \tag{8}$$

$$C_{\max} \ge C_{i,p,l_{ip}} \quad \forall\, i \in I \text{ and } p \in P \tag{9}$$

$$S_{ipj}, C_{ipj}, C_i \ge 0 \quad \forall\, i \in I, p \in P \text{ and } j \in O_{ip} \tag{10}$$

$$Z_{ip}, X_{ipj}, Y_{ipji'p'j'} \in \{0,1\} \quad \forall\, i,i' \in I, p,p' \in P, j \in O_{ip} \text{ and } j' \in O_{i'p'}\ (i \ne i') \tag{11}$$

The objective function, together with constraint (9), represents minimizing the makespan. See Kim and Lee (2023) for more details on the constraints. As explained in Kim and Lee (2023), the problem is NP-hard because the multi-stage ordinary flow shop makespan scheduling problem without queue time limits is NP-hard. Also, in the aspect of scheduling theory, the problem is a new class of flow shop scheduling problem because the scheduling sub-problem is the ordinary flow shop scheduling problem or the reentrant flow shop scheduling problem depending on violating a queue time limit.

4. Three-stage problem

This section considers the three-stage problem with two overlapped queue time limits for each job, i.e. one between stages 1 and 2 and the other between stages 1 and 3. After deriving the properties of the problem, the optimal two-phase branch and bound (TP-B&B) algorithm is explained. Finally, the algorithm is compared with the Gurobi.

4.1. Analysis of the problem

As in the multi-stage problem, the three-stage problem can be decomposed into the job process route selection and the resulting scheduling sub-problems. Based on the decomposition, two properties, one to reduce the search space by specifying a dominant set, and the other to check the feasibility of violating a queue time limit for the jobs

Table 2
Notations.

Sets	
I	set of jobs
P	set of process routes
O_{ip}	set of operations in process route p of job i
Indices	
i	jobs, $i \in I$
p	job process routes, $p \in P$, where $p = 1$ for normal route, and $p > 1$ for abnormal routes
j	ordered operations in each of normal or abnormal process route of a job, $j \in O_{ip}$, where f_{ip} and l_{ip} denote the indices of the first and the last operations in process route p of job i, respectively.
k, k'	operations in each process route of a job, k, $k' \in O_{ip}$
Parameters	
t_{ij}	processing time of operation j of job i or rework setup time of abnormal process route
$Q_{ikk'}$	queue time limit between operations k and k' of job i ($k < k'$)
M	large number
Decision variables	
Z_{ip}	$= 1$ if job i is processed by route p, and 0 otherwise
X_{ipj}	$= 1$ if operation j of job i is processed using process route p, and 0 otherwise
$Y_{ipji'p'j'}$	$= 1$ if operation j in process route p of job i precedes operation j' in process route p' of job i', and 0 otherwise
S_{ipj}	start time of operation j in process route p of job i
C_{ipj}	completion time of operation j in process route p of job i
C_i	completion time of job i

with the normal process route, can be derived.

First, the following proposition implies that the feasible permutation schedules that satisfy the queue time limits dominate the non-permutation ones for the jobs processed by the normal route, which can be used to fathom the unnecessary nodes of the B&B tree.

Proposition 1. *For the jobs processed by the normal process route, the same sequence of the jobs at each stage while satisfying the queue time limits constitutes a dominant set.*

Proof. Consider a schedule in which the jobs in the set I_N and I_A are processed by the normal and abnormal process routes, respectively, where $I = I_N \cup I_A$ and $I_N \cap I_A = \emptyset$. Suppose that there exists a pair of jobs k, $l \in I_N$ with orders $B_B \rightarrow k \rightarrow M_B \rightarrow l \rightarrow A_B$ at the first stage, $B'_B \rightarrow l \rightarrow M'_B \rightarrow k \rightarrow A'_B$ at the second stage, and $B''_B \rightarrow l \rightarrow M''_B \rightarrow k \rightarrow A''_B$ at the third stage, where B_B (A_B) denotes the ordered set of ordinary and possible rework jobs before (after) job k (l) at the first stage, and M_B denotes the ordered set of ordinary and possible rework jobs between jobs k and l at the first stage. At the second stage, B'_B (A'_B) denotes the ordered set of ordinary and possible rework jobs before (after) job l (k), and M'_B denotes the ordered set of ordinary and possible rework jobs between jobs l and k. Similarly, B''_B, M''_B and A''_B are defined as those at the second stage. Note that the ordered sets may be empty and different due to possible reworks. Suppose that the sequences of jobs l and k at the second and the third stages are changed to $B'_A \rightarrow k \rightarrow M'_A \rightarrow l \rightarrow A'_A$ and $B''_A \rightarrow k \rightarrow M''_A \rightarrow l \rightarrow A''_A$, where the ordered sets B'_A (B''_A), M'_A (M''_A) and A'_A (A''_A) are defined as those of the schedule before the changes. Then, the maximum completion times at the second and the third stages cannot be greater than the current ones because the idle times at the two stages cannot be increased by the changes. Therefore, it is sufficient to consider the set of permutation schedules that satisfy the queue time limits for the jobs in I_N, which constitutes a dominant set. ■

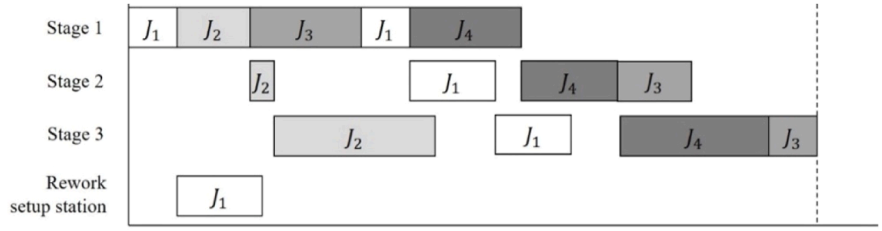
To illustrate the above proposition, consider a problem with four jobs. Fig. 4(a) shows the current schedule in which jobs 2, 3 and 4 are processed by the normal process route. Let $k = 3$ and $l = 4$. Then, as can be seen in the figure, the orders of the two jobs are not same, i.e. $1 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 4$ at the first stage ($B_B = \{1,2\}$, $M_B = \{1\}$, and $A_B = \emptyset$), $2 \rightarrow 1 \rightarrow 4 \rightarrow 3$ at the second stage ($B'_B = \{1,2\}$, $M'_B = \emptyset$, and $A'_B = \emptyset$), and $2 \rightarrow 1 \rightarrow 4 \rightarrow 3$ at the third stage ($B''_B = \{1,2\}$, $M''_B = \emptyset$, and $A''_B = \emptyset$). Suppose that the sequences of the jobs at the second and the third stages are changed to $2 \rightarrow 3 \rightarrow 1 \rightarrow 4$, i.e. $B'_A = B''_A = \{2\}$ and $M'_A = M''_A = \{1\}$ and $A'_A = A''_A = \emptyset$ with a permutation schedule for jobs 2, 3 and 4. Then, the resulting permutation schedule, shown in Fig. 4(b), does not increase the current makespan.

Using the proposition, the following corollary can be derived easily since the problem becomes the ordinary three-stage problem when all jobs are processed by the normal route. See Baker and Trietsch (2013) for the proof.

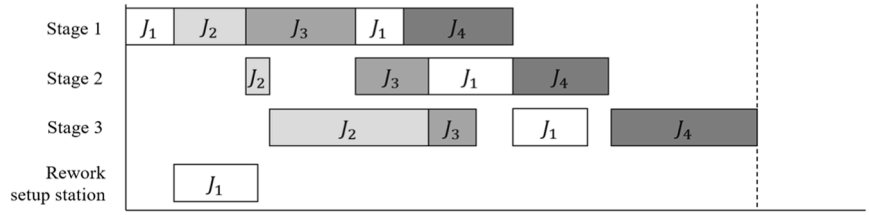
Corollary 1. *Suppose that all jobs are processed by the normal process route. Then, the set of permutation schedules that satisfy the queue time limits constitutes a dominant set.*

The next proposition specifies a condition of violating a queue time limit for each job processed by the normal process route, which can be used to convert an infeasible schedule to a feasible one by inserting the required idle times. In the proposition, D_{ik} denotes the idle time of job i at stage k.

Proposition 2. *Suppose that job r is processed by the normal route. Then,*



(a) Current schedule.



(b) Permutation schedule after interchanging jobs 3 and 4 at stages 2 and 3.

Fig. 4. Proposition 1: example.

if the idle time D_{r1} of job r at the first stage satisfies

$$D_{r1} < \max\{S_{r,1,2} - C_{r,1,1} - Q_{r,1,2},\ S_{r,1,3} - C_{r,1,1} - Q_{r,1,3}\},$$

at least one of the overlapped queue time limits is violated.

Proof. Suppose that $D_{r1} \geq \max\{S_{r,1,2} - C_{r,1,1} - Q_{r,1,2},\ S_{r,1,3} - C_{r,1,1} - Q_{r,1,3}\}$. Then, there exist the following three cases of the relation between $S_{r,1,2} - C_{r,1,1} - Q_{r,1,2}$ and $S_{r,1,3} - C_{r,1,1} - Q_{r,1,3}$.

Case 1: $S_{r,1,2} - C_{r,1,1} - Q_{r,1,2} > S_{r,1,3} - C_{r,1,1} - Q_{r,1,3}$

In this case, $D_{r1} \geq \max\{S_{r,1,2} - C_{r,1,1} - Q_{r,1,2},\ S_{r,1,3} - C_{r,1,1} - Q_{r,1,3}\}$ becomes $D_{r1} < (S_{r,1,2} - C_{r,1,1}) - Q_{r,1,2}$, and hence, $(S_{r,1,2} - C_{r,1,1}) - D_{r1} > Q_{r,1,2}$. This implies that the queue time limit of job r between stages 1 and 2 is violated if $D_{r1} < (S_{r,1,2} - C_{r,1,1}) - Q_{r,1,2}$. Therefore, it is needed to insert an idle time D_{r1} such that $D_{r1} \geq (S_{r,1,2} - C_{r,1,1}) - Q_{r,1,2}$ to obtain a feasible schedule that satisfies the queue time limit.

Case 2: $S_{r,1,2} - C_{r,1,1} - Q_{r,1,2} < S_{r,1,3} - C_{r,1,1} - Q_{r,1,3}$

In this case, $D_{r1} < (S_{r,1,3} - C_{r,1,1}) - Q_{r,1,3}$, and hence $(S_{r,1,3} - C_{r,1,1}) - D_{r1} > Q_{r,1,3}$. This case, similar to the first case, implies that the queue time limit of job r between stages 1 and 3 is violated. Therefore, it is needed to insert an idle time D_{r1} such that $D_{r1} \geq S_{r,1,3} - C_{r,1,1} - Q_{r,1,3}$ to obtain a schedule that satisfies the queue time limit.

Case 3: $S_{r,1,2} - C_{r,1,1} - Q_{r,1,2} = S_{r,1,3} - C_{r,1,1} - Q_{r,1,3}$

This is the same as Cases 1 and 2. ■

4.2. TP-B&B algorithm

The TP-B&B algorithm consists of two iterative phases: (a) generate a candidate set of job process routes; and (b) obtain the resulting optimal schedule. Note that the algorithm is terminated when all possible sets of job process routes are evaluated using the resulting schedules.

4.2.1. Phase 1: generate a set of job process routes

In this phase, a candidate set of job process routes is generated using a two-level tree that represents all possible sets of job process routes. As described in Fig. 3, each job has three process routes, i.e. one normal and two abnormal ones. Let $|I|$ denote the cardinality of job set I. Then, the two-level tree consists of a root and $3^{|I|}$ leaf nodes, where each leaf node represents a set of job process routes. For example, there are 27 leaf nodes for a problem with three jobs, i.e. $|I| = 3$. In the two-level tree, the search is done sequentially from the leftmost leaf node that selects the normal process route for all jobs to the rightmost one that selects the abnormal process route violating the queue time limit between stages 1 and 3 for all jobs.

4.2.2. Phase 2: obtain the optimal schedule under a candidate set of job process routes

For a given set of job process routes, the resulting optimal schedule is obtained using a B&B algorithm that consists of an active schedule generation based branching scheme, an NEH based upper bound, and a modified machine based lower bound. The details are explained below.

Branching

The branching scheme generates all non-dominated feasible schedules for each scheduling sub-problem by modifying the active schedule generation based B&B tree, where the active schedules must be generated because the resulting scheduling sub-problems are the ordinary flow shop scheduling or the reentrant flow shop scheduling problems depending on violating a queue time limit. Note that the active schedules are the smallest dominant set that contains the optimal solutions of the job shop scheduling problems with regular performance measures (Baker & Trietsch, 2013).

In the B&B tree, each node except for the root corresponds to a partial schedule, and each level represents the number of operations fixed in the corresponding partial schedules. Note that the partial schedules in which the jobs with the normal process route have a non-permutation schedule can be fathomed by Proposition 1. Moreover, if a queue time limit of a job with the normal process route is violated for each non-fathomed partial schedule, the required idle time is inserted to the first stage to make it feasible. Formally, according to Proposition 2, if job r with the normal process route violates the queue time limit between stages 1 and 2, the minimum idle time $D_{r1} = (S_{r,1,2} - C_{r,1,1}) - Q_{r,1,2}$ is inserted to stage 1. Similarly, if the job r violates the queue time limit between stages 1 and 3, $D_{r1} = (S_{r,1,3} - C_{r,1,1}) - Q_{r,1,3}$ is inserted to stage 1. Finally, each node of the B&B tree can be fathomed if the lower bound at the node is greater than or equal to the current best upper bound, where the current best upper bound is the smallest one among those obtained by solving the scheduling problems under the current and the previous job process route sets.

The branching starts at the root node of the B&B tree, i.e. level 0, where no operations are fixed. For node selection, the depth-first rule is used. Specifically, if the current node is not fathomed, the next node to be considered is one of its child nodes, i.e. the node with the smallest index. When a node is fathomed, we go back on the path from this node toward the root node until finding the first node with a child node that has not been considered yet, i.e. backtracking. The procedure that generates all non-dominated feasible schedules is given below. In the procedure, PS_l and OP_{ij} denote a partial schedule containing l scheduled operations and operation j of job i, respectively. Note that PS_l corresponds to a node of the B&B tree.

Step 1. Let $l = 0$, $PS_l = \varnothing$, and $S_l =$ set of all operations with no predecessors.

Step 2. Determine $\phi^* = \min_{OP_{ij} \in S_l}\{\phi_{ij}\}$, where ϕ_{ij} denotes the earliest time at which operation $OP_{ij} \in S_l$ can be completed. Also, let m^* denote the machine on which ϕ^* is realized.

Step 3. For each operation $OP_{ij} \in S_l$ that requires machine m^* and for which $\sigma_{ij} < \phi^*$, create a new partial schedule PS_{l+1} in which operation OP_{ij} is added to PS_l and started at time σ_{ij}, where σ_{ij} denotes the earliest time at which operation OP_{ij} can be started. If the created partial schedule contains non-permutation one for the jobs processed by the normal process route, fathom it and go to Step 2. Otherwise, go to Step 4.

Step 4. If the non-fathomed partial schedule is infeasible, obtain a feasible one by inserting the required idle times for the jobs with the normal process route. Update S_l as follows and go to Step 5.

(a) $S_l = S_l \backslash OP_{ij}$;
(b) $S_{l+1} = S_l \cup$ {direct successor of operation $OP_{ij} \in S_l$};
(c) $l = l + 1$

Step 5. Go to Step 2 for each of the partial schedules in PS_{l+1}.

Bounding

At the root node of the B&B tree, an initial upper bound is obtained by modifying the NEH based heuristic of Kim and Lee (2023). Specifically, the job sequence at the first stag is generated using the NEH heuristic after the process route of each job is set to the normal one, and the resulting schedule for the remaining stages is constructed using the priority scheduling method that selects the best one among those generated by priority rules FCFS (first come first served), SOPT (shortest operation processing time), MWKR (most work remaining) and PWR (processing time over remaining work). After a complete schedule is obtained, it is needed to check whether the queue time limits of each job with the normal process route are satisfied. If not, the infeasible schedule is changed into a feasible one using the method explained earlier.

The detailed procedure to obtain an initial upper bound is given below. Note that the upper bound is updated whenever a feasible solution obtained at a leaf node of the B&B tree is better than the current best one obtained so far, where the current best upper bound is updated at each leaf node of the first phase two-level tree.

Step 1. Set the process route of each job to the normal one.

Step 2. Sort all jobs in the non-increasing order of their total processing times, i.e. sums of processing times over all stages.

Step 3. For the first two jobs in the sorted list, evaluate the two job sequences at the first stage after obtaining the complete schedules for all stages using the priority scheduling method and select the better one, where an infeasible complete schedule is converted into a feasible one by inserting required idle time(s).

Step 4. Insert the next job in the sorted list to the best position that gives the minimum makespan, where a tie is broken by selecting the position that gives the smallest completion time on the next stage and each infeasible complete schedule is converted into a feasible one. Do this step repeatedly until all jobs are scheduled.

The lower bound is obtained at each node of the B&B tree. Note that when a lower bound of a partial schedule is larger than or equal to the best upper bound found, the corresponding node is fathomed. For this purpose, the well-known machine based bounds for the ordinary flow shop scheduling problem are used. Let $C_m(PS_l)$ denote the earliest start time at which an unscheduled job can be processed on machine m for partial schedule PS_l. Then, the machine based bounds can be represented as

$$LB_1 = C_1(PS_l) + \sum_{j \notin PS_l} p_{j1},$$
$$LB_2 = C_2(PS_l) + \sum_{j \notin PS_l} p_{j2}, \text{ and}$$
$$LB_3 = C_3(PS_l) + \sum_{j \notin PS_l} p_{j3}.$$

where p_{jm} denotes the processing time of operation j, including the corresponding rework operation, to be processed on stage m. Finally, the lower bound at the B&B node corresponding the partial schedule PS_l is obtained as $\max\{LB_1, LB_2, LB_3\}$. Note that the machine based lower bounds are valid for the resulting scheduling sub-problem since they are calculated without the idle times between consecutive operations on each stage.

4.3. Computational results

The TP-B&B algorithm was compared with Gurobi 11.0.1 for small sized test instances, and the results are reported in this section. The algorithm was coded in C++ and the tests were done on a PC with Intel Core i9 processor at 3.42 GHz and 64GB RAM.

For the experiments, 320 instances, i.e. 20 instances for each combination of four levels of the number of jobs (6, 8, 10, and 12), two levels of tightness of queue time limits (loose and tight) and two levels of the rework setup times (low and high), were generated randomly using a partial real fab data with overlapped queue time limits, provided by the partner of our earlier project. Specifically, the data were generated as follows. In the following, $DU(l, u)$ denotes the discrete uniform distribution with range $[l, u]$.

- Operation times $\sim DU(5,\ 100)$
- Rework setup times $\sim DU(5,\ 20)$ for the low case and $DU(80,\ 100)$ for the high case
- Queue time limits $\sim DU(5,\ 20)$ for the tight case and $DU(80,\ 100)$ for loose case

Test results are summarized in Table 3(a) and (b) that show the numbers of instances that the Gurobi and the TP-B&B algorithm gave the optimal solutions with a time limit of 1 hour and CPU seconds for the cases of loose and tight queue time limits, respectively.

It can be seen from the tables that the TP-B&B algorithm gave 21.5% more optimal solutions than the Gurobi while requiring much less computation times, i.e. 178 and 245 out of 320 test instances for the Gurobi and the TP-B&B algorithm, respectively. In particular, the TP-B&B algorithm gave optimal solutions for some test instances with 12 jobs. This shows the effectiveness of the decomposition based two-phase approach proposed in this study. In particular, the TP-B&B algorithm performed better for the case of loose queue time limits because the dominance property of permutation schedules for the jobs processed by the normal process route gets more effective. Finally, it can be found that the problem complexity decreases as the rework setup times increases because tighter lower bounds may improve the computational efficiency.

Table 3
Test results for the TP-B&B algorithm.

(a) Loose queue time limits.

Rework setup times	Number of jobs	Gurobi		TP-B&B	
		N_{opt}[1]	CPU[2]	N_{opt}[1]	CPU[2]
Low	6	20	8.1 (2.7, 19.7)	20	2.7 (0.2, 6.4)
	8	20	151.4 (37.5, 322.0)	20	30.1 (0.9, 87.1)
	10	1	927.2	17	1627.4 (189.7, 3352.1)
	12	-	-	7	1442.9 (0.1, 2457.8)
High	6	20	3.9 (1.2, 6.5)	20	2.6 (0.1, 4.1)
	8	20	192.7 (32.0, 515.2)	20	86.7 (5.1, 249.7)
	10	11	1987.8 (987.2, 3235.8)	19	452.7 (48.9, 1405.4)
	12	-	-	9	1687.1 (42.2, 2319.9)

(b) Tight queue time limits.

Rework setup times	Number of jobs	Gurobi		TP-B&B	
		N_{opt}[1]	CPU[2]	N_{opt}[1]	CPU[2]
Low	6	20	6.8 (2.7, 12.8)	20	1.4 (0.3, 4.1)
	8	13	183.3 (39.8, 362.1)	20	69.7 (23.8, 179.7)
	10	3	2471.3 (2241.8, 2241.8)	11	1341.4 (37.7, 3379.1)
	12	-	-	3	2264.1 (1781.1, 2697.2)
High	6	20	3.7 (1.2, 7.0)	20	2.2 (0.1, 5.2)
	8	20	289.8 (35.9, 470.1)	20	101.7 (8.2, 318.2)
	10	10	1586.4 (138.3, 2817.9)	17	881.7 (211.3, 1662.5)
	12	-	-	2	2680.9 (2589.9, 2771.9)

[1] Number of instances that the optimal solutions were obtained out of 20 instances within 1 h.
[2] Average CPU seconds for the instances that gave the optimal solutions (minimum and maximum in parenthesis).

5. Multi-stage problem

This section presents the two-phase variable neighborhood search (TP-VNS) algorithms for the multi-stage problem. In general, the VNS algorithm is a trajectory-based meta-heuristic that improves an initial solution using shaking to escape the local optimums and local search improvement to converge the shaking solutions towards the global optimum by changing different neighborhood structures in a systematic way. Refer to Mladenović and Hansen (1997) for the basics of the VNS

algorithm.

5.1. Solution algorithms

As in the three-stage problem, the multi-stage problem can be decomposed into the job process route selection and the resulting scheduling sub-problems. Based on the decomposition property, two TP-VNS algorithms are proposed.

5.1.1. Basic TP-VNS algorithm

As can be seen in Fig. 5, the basic TP-VNS algorithm consists of four steps: (a) obtaining an initial solution; (b) generating shaking solutions using the different neighborhood structures; (c) improving the shaking solutions by local search improvement; and (d) checking the termination condition. Unlike the previous algorithms of Kim and Lee (2023) in which the initial solution is improved by changing the schedules without considering job process routes, the TP-VNS algorithms change both the job process routes and the resulting schedules at the same time with different neighborhood structures.

In the algorithm, a solution is represented as

$$\boldsymbol{S} = \left((x_{[1]}, z_{[1]}), (x_{[2]}, z_{[2]}), \ldots, (x_{[|I|]}, z_{[|I|]})\right),$$

where $x_{[u]}$ and $z_{[u]}$ denote the indices of the job at the uth position of the sequence at the first stage and the process route of job $x_{[u]}$. For example, a solution $((4,1),(1,1),(3,2),(5,3),(2,2),(6,3))$ for a problem with six jobs implies that the job sequence at the first stage is 4→1→3→5→2→6, and jobs 1 and 4 are processed by the normal process route 1, jobs 2 and 3 are processed by the abnormal process route 2, and jobs 5 and 6 are processed by the abnormal process route 3, respectively. Then, a complete schedule is constructed using the priority scheduling method used to obtain the initial upper bound of the TP-B&B algorithm while considering the additional rework setups and operations for the jobs with abnormal process routes. Recall that if the resulting complete schedule is infeasible for a job with the normal process route, it is converted to a feasible one by inserting the required idle time to the first stage.

5.1.1.1. Obtaining an initial solution. An initial solution is obtained using the method to obtain the initial upper bound of the TP-B&B algorithm explained earlier.

5.1.1.2. Shaking. The shaking is done to escape from local optimums by selecting a neighborhood structure from given neighborhood structures and generating a shaking solution using the selected structure randomly,

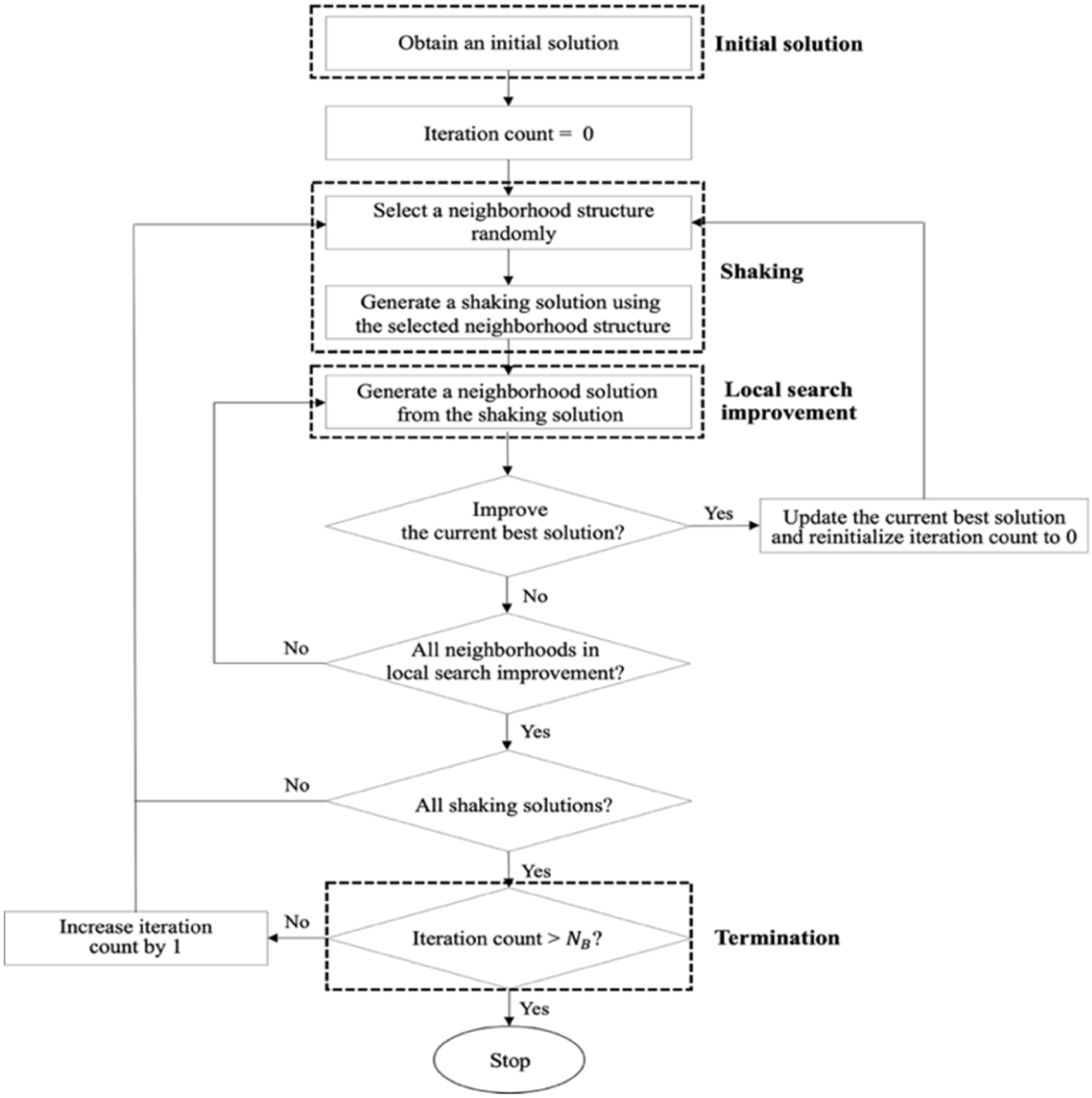


Fig. 5. An overview of the basic TP-VNS algorithm.

where the random selection, called the random neighborhood change step in the literature, is done to avoid cycling. Formally, let Γ denote the set of neighborhood structures. Then, a shaking solution $\boldsymbol{S}'$ is generated as $\boldsymbol{S}' \in N'(\boldsymbol{S})$ randomly, where $N'(\boldsymbol{S})$ denotes the set of neighborhood solutions generated from the solution $\boldsymbol{S}$ using the random neighborhood structure $N' \in \Gamma$. Note that it is unnecessary to check the feasibility of the complete schedule obtained from the shaking solution $\boldsymbol{S}'$ because the shaking is done to escape from local optimums, not to improve the current best complete schedule.

In this study, the following neighborhood structures that change both the job sequence at the first stage and the job process routes are proposed.

Sequence insertion & route insertion (*SI&RI*). Select two jobs $x_{[u']}$ and $x_{[u'']}$ at positions $u' \le |I|/2$ and $u'' > |I|/2$ randomly, and move job $x_{[u']}$ to position $u''-1$ after moving jobs $x_{[u'+1]}, x_{[u'+2]}, \ldots, x_{[u''-1]}$ to positions $u', u'+1, \ldots, u''-2$, respectively. Then, select two job process routes $z_{[v']}$ and $z_{[v'']}$ at positions v' and v'' ($v' \neq v''$) randomly and move process route $z_{[v']}$ to position $v''-1$ using the same method. See Fig. 6(a) for an example of the SI&RI method with $(u', u'') = (2,5)$ and $(v', v'') = (3,5)$.

Sequence insertion & route swap (*SI&RS*). Same as SI&RI except the route swap method, i.e. select two job process routes $z_{[v']}$ and $z_{[v'']}$ ($v' \neq v''$) randomly, and interchange them. See Fig. 6(b) for the earlier example with $(v', v'') = (1,3)$.

Sequence insertion & route reverse (*SI&RR*). Same as SI&RI except the route reverse method, i.e. select two job process routes $z_{[v']}$ and $z_{[v'']}$ ($v' \neq v''$) randomly, and move process routes $z_{[v'+1]}, z_{[v'+2]}, \ldots, z_{[v''-1]}$ to positions $v''-1, v''-2, \ldots, v'+1$, respectively. See Fig. 6(c) for the earlier example with $(v', v'') = (1,6)$.

Sequence swap & route insertion (*SS&RI*). Same as SI&RI except the sequence swap method, i.e. select the two jobs $x_{[u']}$ and $x_{[u'']}$ with $u' \le |I|/2$ and $u'' > |I|/2$ randomly, and interchange them.

Sequence swap & route swap (*SS&RS*). Same as SI&RS except the sequence swap method.

Sequence swap & route reverse (*SS&RR*). Same as SI&RR except the sequence swap method.

Sequence reverse & route insertion (*SR&RI*). Same as SI&RI except that the sequence reverse method, i.e. select two jobs $x_{[u']}$ and $x_{[u'']}$ with $u' \le |I|/2$ and $u'' > |I|/2$ randomly, and move jobs $x_{[u'+1]}, x_{[u'+2]}, \ldots, x_{[u''-1]}$ to positions $u''-1, u''-2, \ldots, u'+1$, respectively.

Sequence reverse & route swap (*SR&RS*). Same as SI&RS except the sequence reverse method.

Sequence reverse & route reverse (*SR&RR*). Same as SI&RR except the sequence reverse method.

5.1.1.3. Local search improvement. The local search is done to improve the complete schedule obtained from the current best solution $\boldsymbol{S}^*$ using the shaking solution $\boldsymbol{S}'$. Specifically, a solution $\boldsymbol{S}'' \in N''(\boldsymbol{S}')$ is generated from $\boldsymbol{S}'$ using a random neighborhood structure $N'' \in \Gamma$, and a complete schedule for the generated solution $\boldsymbol{S}''$ is obtained using the priority scheduling method explained earlier. As explained above, if the complete schedule is infeasible for the jobs with the normal process route, it is converted to a feasible one. Then, if the resulting complete schedule improves the current best one, the current best solution $\boldsymbol{S}^*$ is updated, and the next shaking is done using the updated best solution after the iteration count is reinitialized to 0. Otherwise, the local search is done

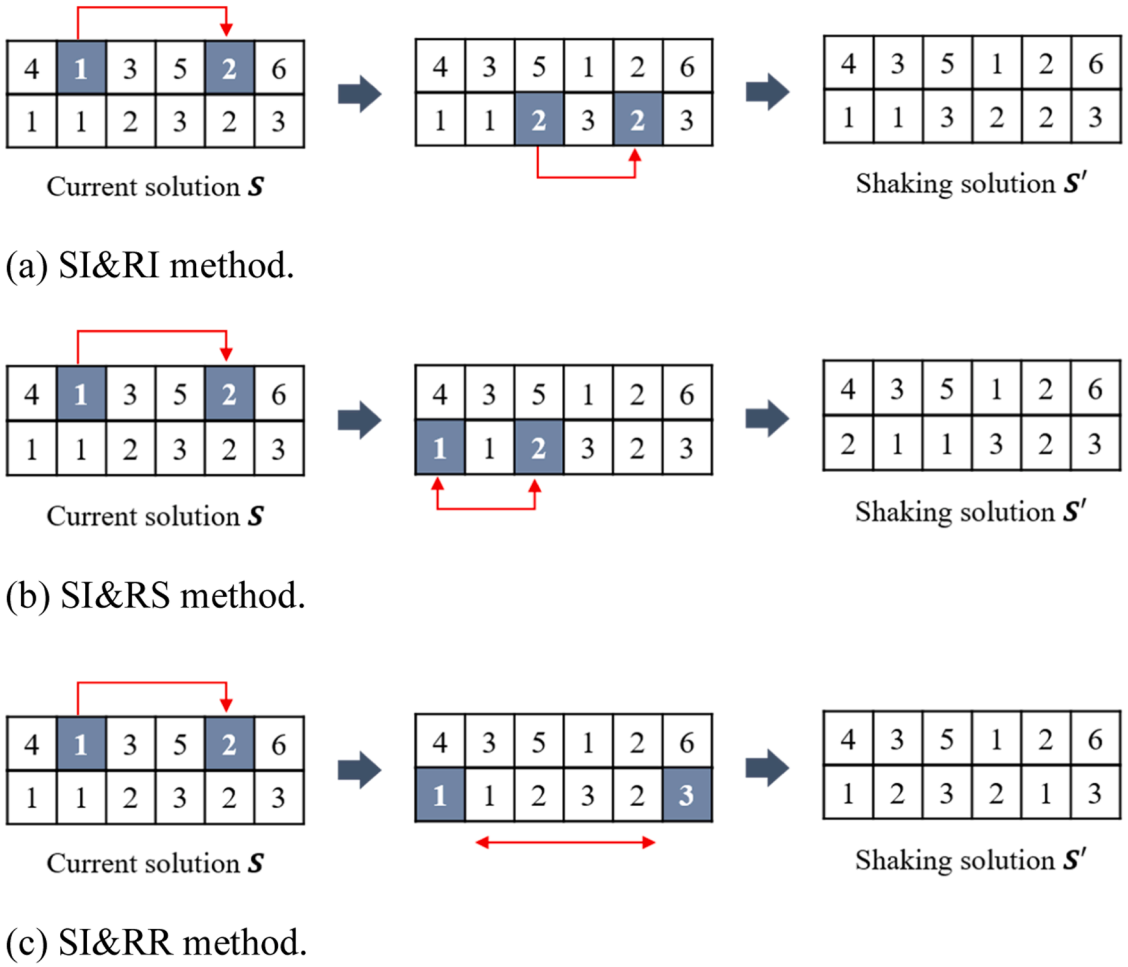


Fig. 6. Examples of the neighborhood structures.

repeatedly using another neighborhood structure until all the neighborhood structures are used. Finally, if all neighborhood structures are used in the local search improvement, the local search is done for another shaking solution generated using the neighborhood structures not used in the current shaking until all the neighborhood structures are used. Note that the iteration count is increased by 1 when no improved solution can be obtained by the shaking and local search improvement steps.

5.1.1.4. Termination. The algorithm is terminated when there is no improvement for N_B consecutive iterations of shaking and local search improvement.

5.1.2. General TP-VNS algorithm

The general TP-VNS algorithm improves the basic one using a variable neighborhood descent (VND) method that uses the neighborhood structures according to a given order in the shaking and the local search improvement and generates n_{ls} solutions under each neighborhood structure in the local search improvement. See Hansen et al. (2006) for the advantages of the VND method. As in the basic one, the general algorithm is terminated when there is no improvement for N_G consecutive iterations.

5.2. Computational results

To test the performances of the two TP-VNS algorithms proposed in this study, they were compared with the previous GVNS algorithm of Kim and Lee (2023), and the results are reported in this section. In addition, two sensitivity analyses are done to examine: (a) the performances of the GVNS and the general TP-VNS algorithms under different levels of rework setup times and queue time limits; and (b) the performance of the general TP-VNS algorithm according to the changes of the parameters. The algorithms were coded in Python, and the tests were done on a PC with Intel Core i9 processor at 3.42 GHz and 64 GB RAM.

5.2.1. Preliminary test

Before the main tests, a preliminary test was done for the general TP-VNS algorithm to set the order of the nine neighborhood structures in each of the shaking and the local search improvement steps as well as the parameters n_{ls} (number of solutions generated by a neighborhood structure in the local search improvement step) and N_G (termination condition). For this test, 30 instances, i.e. 5 instances for each of 6 combinations of three levels for the number of jobs (20, 60, and 100) and two levels for the number of stages (5 and 7), were generated randomly using the real fab data provided by the project partner. The detailed data were generated as follows.

- Operation times $\sim DU(5,\ 100)$
- Rework setup times $\sim DU(5,\ 20)$
- Queue time limits $\sim DU(5,\ 20)$
- Number of queue time limits for each job $\sim DU(2,\ N_s)$, where N_s denotes the number of stages for instance s.

Due to a huge number of possible combinations of the neighborhood orders in the shaking and the local search improvement steps, the same orders in both steps were tested after the parameters n_{ls} and N_G were set to 3 and 50, respectively. To compare the algorithms with different orders, the relative performance ratio (RPR) was used because the optimal solutions could not be obtained in the reasonable time, where the RPR of an algorithm with order o for an instance is calculated as

$$100\cdot(M_o - M_{best})/M_{best},$$

where M_o and M_{best} denote the makespan obtained from the algorithm with order o and the best makespan among those obtained from the algorithms with all the orders. To find the best one among all possible orders, a Duncan's multiple range test was done using the overall average RPR values, and the order of SI&RS - SR&RI - SI&RR - SS&RI - SI&RI - SS&RS - SS&RR - SR&RS - SR&RR was selected. Refer to Kim and Lee (2023) for the best order of the previous GVNS algorithm.

Then, under the best neighborhood order, the parameters n_{ls} and N_G were set using a full factorial design method that selects the best one among 12 combinations of three levels of n_{ls} (3, 4, and 5) and four levels of N_G (30, 40, 50, and 60). Test results are shown in Fig. 7 that shows the average RPR values and average CPU seconds of the algorithm with different parameter combinations. As can be seen in the figure, the combination (3, 60) gave the smallest overall average RPR values with a reasonable amount of computation times. In summary, the general TP-VNS algorithm with the best neighborhood order was tested under the parameters n_{ls} and N_G of 3 and 60, respectively. Note that the termination condition of the basic TP-VNS algorithm was set to the same as the general TP-VNS algorithm, i.e. $N_B = 60$, and the parameters of the previous GVNS algorithm were also set to those of the general TP-VNS algorithm for fair comparisons.

5.2.2. Main tests

The first main experiment was done to show the absolute performance of the three VNS algorithms, i.e. GVNS, basic TP-VNS, and general TP-VNS, for small sized test instances. For this test, 60 random instances, i.e. 10 instances for each of 6 combinations of two levels for the number of jobs (6 and 8) and three levels for the number of stages (5, 7, and 10), were generated using the data generation method explained earlier. The performance measures used are: (a) percentage optimality gaps from the optimal solution values; and (b) CPU seconds, where the optimality gap of algorithm a for an instance is calculated as

$$100\cdot(M_a - M_{opt})/M_{opt},$$

where M_a and M_{opt} denote the makespan obtained from algorithm a and the optimal makespan obtained by solving the model **[P]** using Gurobi 11.0.1 with a time limit of 1 hour, respectively.

Test results are summarized in Table 4 that shows the numbers of instances that gave the optimal solutions, percentage optimality gaps, and CPU seconds. It can be seen from the table that the overall average gaps of the GVNS, the basic TP-VNS, and the general TP-VNS algorithms are 2.0%, 1.3%, and 0.8 %, respectively, while the Gurobi could not give the optimal solutions for all the test instances with 8 jobs. This shows that the three algorithms can give near optimal solutions for the small sized test instances while requiring much less computation times than the Gurobi. Among them, the TP-VNS algorithms were better than the previous GVNS algorithm in overall averages. Specifically, the basic and the general TP-VNS algorithms gave 0.7% and 1.2% improvements over the GVNS algorithm while requiring much less computation times. In fact, paired t-tests were done using the percentage optimality gaps, and the results showed that the TP-VNS algorithms are statistically better than the GVNS algorithm with a significance level of 0.05. This shows the effectiveness of the two-phase approach proposed in this study. Also, due to the enlarged search spaces of the VND method, the general TP-VNS algorithm improved the basic TP-VNS algorithm, but there were no statistical difference between them because the number of scheduling decision points of the test instances is quite small. Finally, the general TP-VNS algorithm gave the solutions within 50 s for all the test instances.

The second main test was done to show the relative performances of the algorithms for medium-to-large sized test instances. For this test, 180 instances, i.e. 10 instances for each of 18 combinations of six levels for the number of jobs (10, 20, 40, 60, 80, and 100) and three levels for the number of stages (5, 7, and 10), were generated using the data generation method explained earlier. To evaluate the result, the relative performance ratio (RPR) was used because the Gurobi could not give the optimal solutions.

Test results are summarized in Table 5 that shows the average RPR

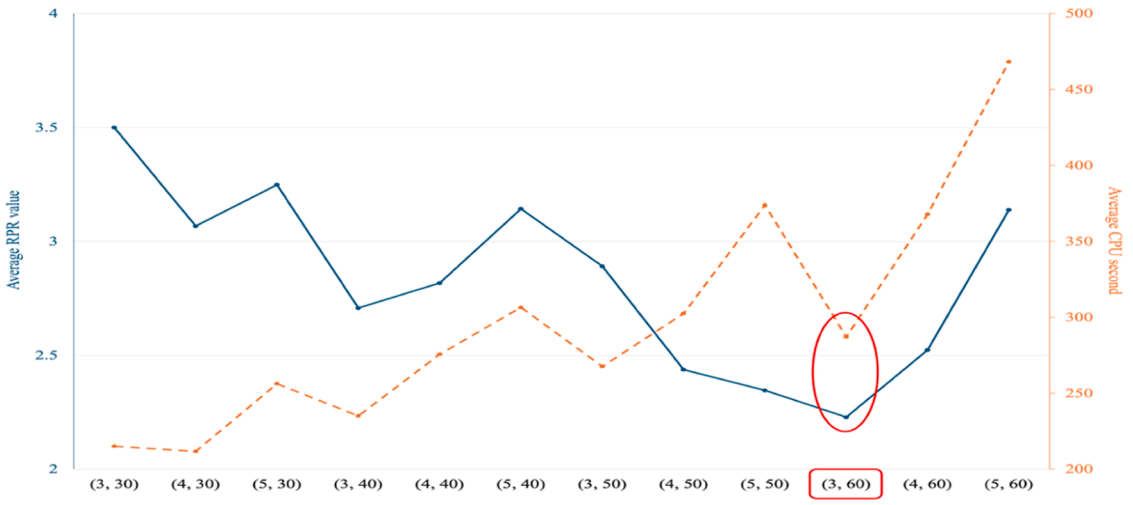


Fig. 7. Results for setting the parameters of the general TP-VNS algorithm.

Table 4
Results for the small sized test instances.

Number of jobs	Number of stages	Gurobi		GVNS			Basic TP-VNS			General TP-VNS		
		N_{opt}[1]	CPU_G[2]	N_{opt}[1]	APG[4]	CPU_V[3]	N_{opt}[1]	APG[4]	CPU_V[3]	N_{opt}[1]	APG[4]	CPU_V[3]
6	5	10	27.3	6	1.7 (0.0, 7.9)	14.7	8	1.5 (0.0, 5.2)	5.8	8	1.1 (0.0, 4.9)	12.9
	7	10	81.2	3	1.6 (0.0, 3.9)	15.1	4	1.2 (0.0, 3.2)	8.3	5	0.9 (0.0, 3.1)	14.8
	10	10	252.9	2	1.9 (0.0, 3.8)	21.6	3	1.5 (0.0, 3.8)	12.1	4	1.2 (0.0, 4.2)	17.8
8	5	9	728.1	5	1.4 (0.0, 4.5)	19.5	6	0.6 (0, 2.1)	10.3	7	0.3 (0.0, 1.3)	17.6
	7	5	1632.8	1	3.6 (0.0, 6.8)	24.1	3	3.1 (0.0, 5.8)	12.4	3	1.2 (0.0, 2.9)	21.8
	10	2	2602.9	1	1.8 (0.0, 2.7)	38.2	1	0.4 (0.0, 1.9)	18.7	1	0.2 (0.0, 0.5)	36.4
Average		7.7		3	2.0		4.2	1.3		4.6	0.8	

[1] Number of instances that the optimal solutions were obtained out of 10 instances within 1 hour.
[2] Average CPU seconds for the instances that gave the optimal solutions.
[3] Average CPU seconds out of 10 instances.
[4] Average percentage optimality gaps out of 10 instances (minimum and maximum in parenthesis).

values and CPU seconds of the three VNS algorithms. As in the test results for the small sized instances, the two TP-VNS algorithms outperformed the GVNS algorithm in overall averages, i.e. 1.2% and 2.9% improvements in RPR for the basic and the general TP-VNS algorithms, respectively. In particular, the general TP-VNS algorithm gave much better solutions than the others. This implies that the effectiveness of the VND method gets higher as the problem size increases. To show the statistical differences among the algorithms, paired *t*-tests were done using the RPR values, and the results showed that the TP-VNS algorithms are statistically better than the GVNS algorithm with a significance level of 0.05. However, the best general TP-VNS algorithm required longer computation times than the others due to the enlarged search space.

Finally, sensitivity analyses were done to examine the performances of the GVNS and the general TP-VNS algorithms under different levels of rework setup times and queue time limits as well as the performance of the general TP-VNS algorithm according to the changes of the parameters n_{ls} and N_G. For this purpose, 120 instances, i.e. 5 instances for each of 24 combinations of three levels for the number of jobs (20, 60, and 100), two levels for the number of stages (5 and 7), two levels of rework setup times (low and high), and two levels of tightness of queue time limits (tight and loose), were generated using data generation method explained earlier except for the following data.

- Queue time limits $\sim DU(5,\ 20)$ for the tight case and $DU(80,\ 100)$ for the loose case
- Rework setup times $\sim DU(5,\ 20)$ for the low case and $DU(80,\ 100)$ for the high case

Fig. 8 shows the average RPR values of the two algorithms under the four combinations of rework setup times and queue time limits, i.e. low/tight in Fig. 8(a); low/loose in Fig. 8(b); high/tight in Fig. 8(c); and high/loose in Fig. 8(d), when the parameters n_{ls} and N_G were set to the same as the main experiments, i.e. $n_{ls} = 3$ and $N_G = 60$. It can be seen from the figures that the differences in the average RPR values are larger when the queue time limits are tight. This is because tighter queue time limits increase the possibility of selecting abnormal process routes, and the general TP-VNS algorithm is more effective to handle the abnormal process routes as well as the normal process routes with inserted idle times. Also, due to the same reason, the differences in the average RPR values are larger when the rework setup times are high. Note that the scheduling sub-problem is similar to the ordinary flow shop scheduling problem when the queue time limits are loose.

Fig. 9 shows the average RPR values (solid lines) and average CPU seconds (dotted lines) of the general TP-VNS algorithm under 15 combinations of five levels of n_{ls} (3, 5, 7, 10, and 20) and three levels of N_G (50, 100, and 200) for each of the four combinations of rework setup

Table 5
Results for the medium-to-large sized test instances.

Number of jobs	Number of stages	GVNS		Basic TP-VNS		General TP-VNS	
		ARPR[1]	CPU[2]	ARPR[1]	CPU[2]	ARPR[1]	CPU[2]
10	5	3.1 (0.0, 6.9)	26.7	1.2 (0.0. 2.9)	12.7	0.0 (0.0, 0.0)	27.4
	7	2.3 (0.0, 5.1)	37.1	1.2 (0.0, 3.5)	17.3	0.0 (0.0, 0.0)	40.1
	10	1.6 (0.0, 3.2)	42.8	1.4 (0.0, 2.9)	19.2	0.0 (0.0, 0.0)	44.9
20	5	3.0 (0.0, 8.1)	64.3	2.5 (0.0, 4.4)	29.8	0.1 (0.0, 1.3)	69.7
	7	2.5 (0.0, 5.3)	75.1	2.3 (0.0, 3.6)	34.2	0.1 (0.0, 1.1)	80.1
	10	2.6 (0.0, 4.6)	112.9	2.5 (0.0, 4.7)	54.7	0.0 (0.0, 0.9)	122.4
40	5	3.8 (0.0, 6.1)	169.3	1.1 (0.0, 3.9)	72.4	0.4 (0.0, 1.8)	183.6
	7	4.8 (2.3, 6.9)	184.1	1.7 (0.0, 5.6)	131.8	0.2 (0.0, 2.1)	201.4
	10	3.3 (1.8, 9.3)	227.3	2.3 (0.5, 6.7)	144.1	0.1 (0.0, 1.9)	251.7
60	5	4.5 (1.1, 7.6)	231.6	2.1 (0.0, 4.3)	155.3	0.0 (0.0, 0.3)	241.4
	7	3.5 (1.6, 6.5)	334.8	2.5 (0.2, 4.8)	183.8	0.0 (0.0, 0.0)	347.8
	10	2.8 (0.8, 5.3)	411.7	1.2 (0.0, 4.4)	329.2	0.0 (0.0, 0.0)	408.4
80	5	2.5 (1.1, 3.6)	429.1	1.7 (0.0, 3.3)	345.2	0.3 (0.0, 1.6)	420.7
	7	2.8 (1.3, 5.8)	541.8	1.0 (0.0, 3.1)	378.8	0.1 (0.0, 1.1)	571.7
	10	3.2 (1.3, 5.5)	644.5	2.1 (0.7, 3.8)	469.5	0.0 (0.0, 0.5)	627.4
100	5	3.5 (1.6, 7.8)	685.7	2.5 (0.0, 5.1)	501.4	0.2 (0.0, 1.3)	678.6
	7	2.4 (1.3, 4.4)	789.4	1.5 (0.4, 3.1)	584.8	0.2 (0.0, 1.1)	823.8
	10	2.9 (1.2, 5.6)	897.3	1.8 (0.0, 2.9)	741.1	0.6 (0.0, 2.1)	931.5
Average		3.1		1.9		0.2	

[1] Average relative performance ratio out of 10 instances (minimum and maximum in parenthesis).
[2] Average CPU seconds out of 10 instances.

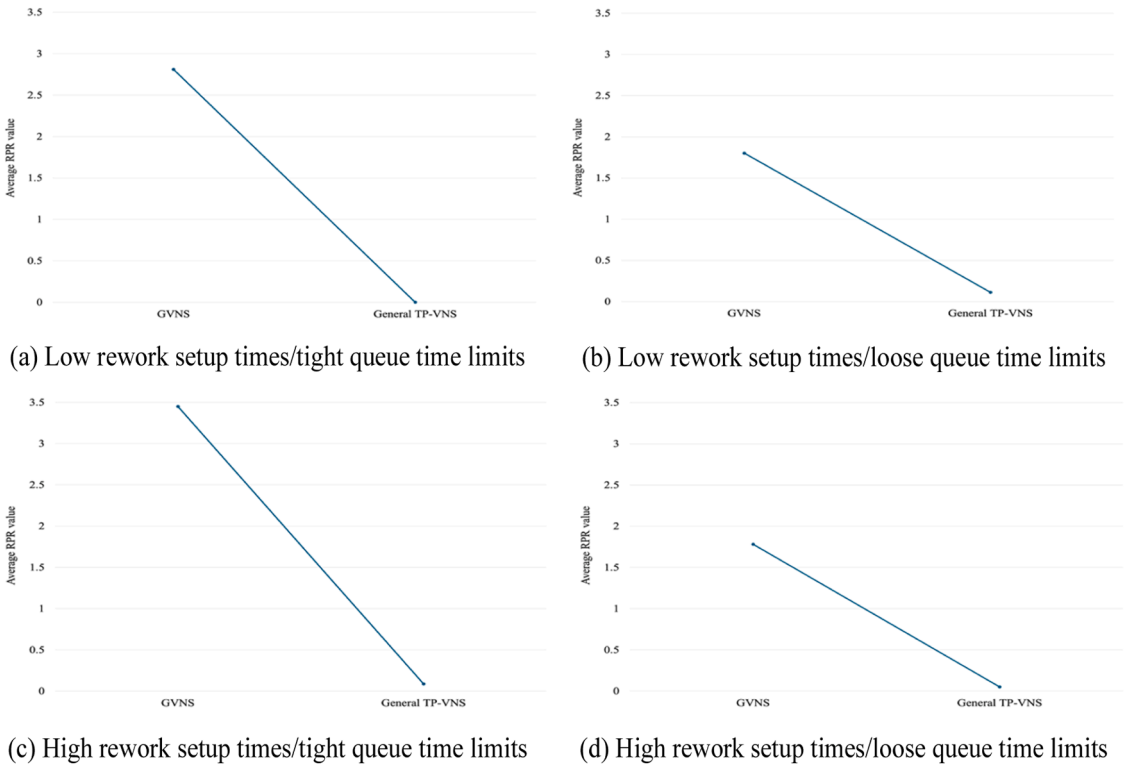
times and queue time limits. As can be seen in the figures, the average RPR values tend to decrease while the CPU seconds increase significantly as the parameter values increase. This implies that the benefits of large parameter values are marginal with respect to the solution quality and computation time. Therefore, it is needed to set the parameters appropriately in order to obtain the good solutions within a reasonable amount of computational times.

6. Concluding remarks

This study considered the problem of determining the start times of jobs at each stage and rework setups and operations, if occur, for flow shops with reworks under overlapped queue time limits. After decomposing the problem into the job process route selection and the resulting flow shop or reentrant flow shop scheduling sub-problems, two types of two-phase algorithms were proposed. For the three-stage makespan problem, an optimal TP-B&B algorithm was developed that consists of generating all possible job process routes using a two-level tree and finding the resulting optimal flow shop or reentrant flow shop schedules using a B&B algorithm. Computational results showed that the algorithm gives more optimal solutions than the Gurobi while requiring much less computation times for the small sized test instances. In addition, for the multi-stage makespan problem, two TP-VNS algorithms, i.e. basic one that solves each sub-problem using a shaking and local search improvement method with different neighborhood structures and general one with a VND method, were proposed. Computational results showed that the TP-VNS algorithms outperform the existing GVNS algorithm, and give near optimal solutions for small sized test instances, and the general TP-VNS algorithm improves the basic algorithm significantly. This shows the effectiveness of the two-phase approach proposed in this study.

This study has a theoretical contribution of developing the first optimal algorithm for the three-stage flow shop makespan scheduling problem with reworks under overlapped queue time limits as well as the improved heuristic algorithms for the multi-stage problem. Note that the optimal algorithm reduces the search space by the property that the permutation schedules for the jobs with the normal route constitutes a dominance set while converting an infeasible schedule that violates a queue time limit of a job with the normal process route to a feasible one as well as updating the best upper bound using those obtained by solving the resulting scheduling sub-problems. Also, in the managerial aspect, the optimal and heuristic algorithms proposed in this study can be used as a core APS methodology for a variety of manufacturing systems with reworks under overlapped queue time limits.

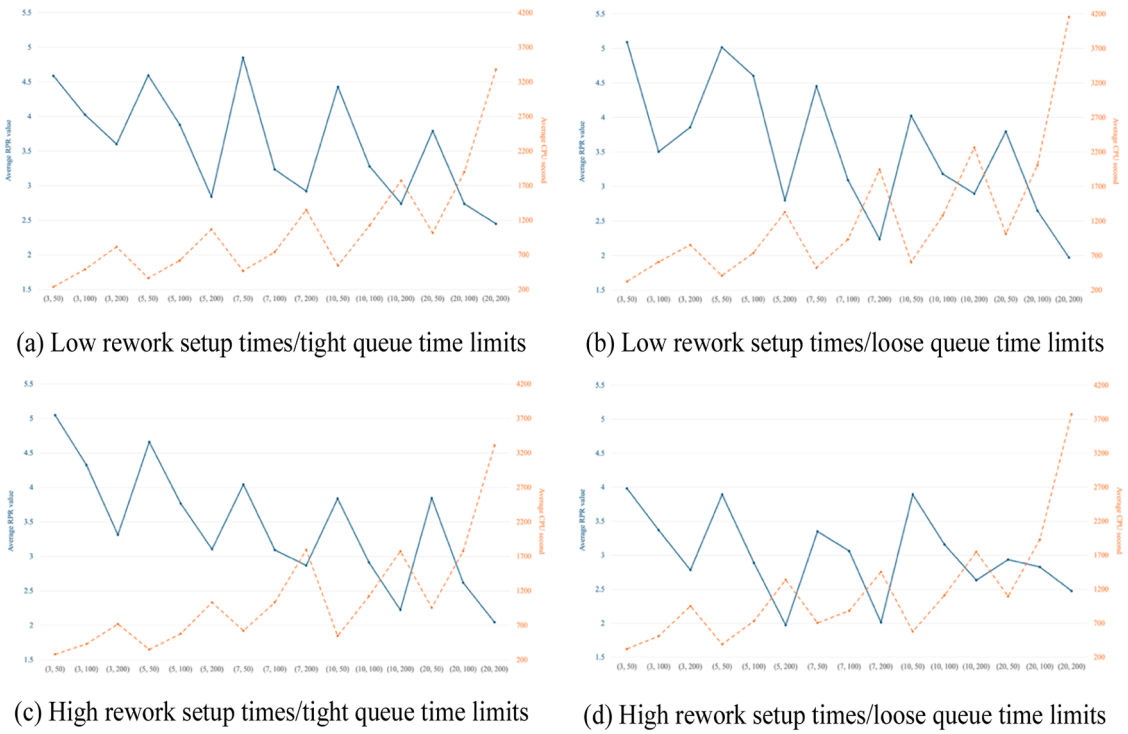
As an early study on flow shop scheduling with reworks under overlapped queue time limits, this study can be extended in several directions that overcome the limitations. First, it is a challenge to make the mixed integer programming model proposed in this study be tighter using problem-specific valid inequalities and other solution encoding schemes. Second, the search spaces of the TP-B&B algorithm can be reduced by developing more efficient branching and bounding schemes, and the TP-VNS algorithms can be improved by more sophisticated neighborhood structures. Third, the static and deterministic problem considered in this study can be extended to the ones with non-zero times,



(a) Low rework setup times/tight queue time limits (b) Low rework setup times/loose queue time limits

(c) High rework setup times/tight queue time limits (d) High rework setup times/loose queue time limits

Fig. 8. Results for the sensitive analysis on rework setup times and queue time limits.



(a) Low rework setup times/tight queue time limits (b) Low rework setup times/loose queue time limits

(c) High rework setup times/tight queue time limits (d) High rework setup times/loose queue time limits

Fig. 9. Results for the sensitive analysis on the parameter changes of the general TP-VNS algorithm.

multiple objectives, and dynamic/stochastic events such as urgent jobs, machine breakdowns, and so on. For these extensions, various machine learning techniques that can be used in real-time may be helpful. In addition, the problem can be extended to hybrid flow shop scheduling problem with multiple machines at each stage. Finally, a case study is needed to show the practical applicability of the solution algorithms proposed in this study.

CRediT authorship contribution statement

Hyeon-Il Kim: Writing – original draft, Validation, Software, Methodology, Formal analysis, Data curation. **Dong-Ho Lee:** Writing –

review & editing, Writing – original draft, Supervision, Resources, Methodology, Investigation, Funding acquisition.

Declaration of competing interest

Dong-Ho Lee reports financial support was provided by National Research Foundation of Korea. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the National Research Foundation of Korea (NRF) grant funded by Korea government (MSIT). (No. 2022R1F1A1066641, Title: Hybrid Flow Shop Scheduling with Reworks under Overlapped Queue Time Limits: Mathematical Models and Predictive/Reactive Algorithms)

References

Akkerman, R., van Donk, D. P., & Gaalman, G. (2007). Influence of capacity-and time-constrained intermediate storage in two-stage food production systems. *International Journal of Production Research, 45*(13), 2955–2973.

An, Y.-J., Kim, Y.-D., & Choi, S.-W. (2016). Minimizing makespan in a two-machine flowshop with a limited waiting time constraint and sequence-dependent setup times. *Computers & Operations Research, 71*, 127–136.

Baker, K. R., & Trietsch, D. (2013). *Principles of sequencing and scheduling*. John Wiley & Sons.

Chen, J. S., & Yang, J. S. (2006). Model formulations for the machine scheduling problem with limited waiting time constraints. *Journal of Information and Optimization Sciences, 27*(1), 225–240.

de Athayde Prata, B., de Abreu, L. R., & Fernandez-Viagas, V. (2025). A systematic review of permutation flow shop scheduling with due-date-related objectives. *Computers & Operations Research, 177*, Article 106989.

Fernandez-Viagas, V. (2025). Transportation and delivery in flow-shop scheduling problems: A systematic review. *European Journal of Operational Research, 325*(1), 1–19.

Gicquel, C., Hege, L., Minoux, M., & van Canneyt, W. (2012). A discrete time exact solution approach for a complex hybrid flow-shop scheduling problem with limited-wait constraints. *Computers & Operations Research, 39*(3), 629–636.

González-Neira, E., Montoya-Torres, J., & Barrera, D. (2017). Flow-shop scheduling problem under uncertainties: Review and trends. *International Journal of Industrial Engineering Computations, 8*(4), 399–426.

Ham, M. (2012). Integer programming-based real-time dispatching (i-RTD) heuristic for wet-etch station at wafer fabrication. *International Journal of Production Research, 50*(10), 2809–2822.

Ham, M., Lee, Y.-H., & An, J.-H. (2011). IP-based real time dispatching for two machine batching problem with time window constraints. *IEEE Transactions on Automation Science and Engineering, 8*(3), 589–597.

Han, J.-H., & Lee, J.-Y. (2021). Heuristics for a two-stage assembly-type flow shop with limited waiting time constraints. *Applied Sciences, 11*(23), Article 11240.

Han, J.-H., & Lee, J.-Y. (2023a). Genetic algorithm-based approach for makespan minimization in a flow shop with queue time limits and skipping jobs. *Advances in Production Engineering & Management, 18*(2), 152–162.

Han, J.-H., & Lee, J.-Y. (2023b). Scheduling for a flow shop with waiting time constraints and missing operations in semiconductor manufacturing. *Engineering Optimization, 55*(10), 1742–1759.

Hansen, P., Mladenović, N., & Urošević, D. (2006). Variable neighborhood search and local branching. *Computers & Operations Research, 33*(10), 3034–3045.

Harjunkoski, I., & Grossmann, I. E. (2001). A decomposition approach for the scheduling of a steel plant production. *Computers & Chemical Engineering, 25*(11–12), 1647–1660.

Hong, T. Y., Chien, C. F., Wang, H. K., & Guo, H. Z. (2018). A two-phase decoding genetic algorithm for TFT-LCD array photolithography stage scheduling problem with constrained waiting time. *Computers & Industrial Engineering, 125*, 200–211.

İnce, N., Deliktaş, D., & Selvi, İ. H. (2024). A comprehensive literature review of the flowshop group scheduling problems: Systematic and bibliometric reviews. *International Journal of Production Research, 62*(12), 4565–4594.

Jeong, B., Han, J.-H., & Lee, J.-Y. (2021). Metaheuristics for a flow shop scheduling problem with urgent jobs and limited waiting times. *Algorithms, 14*(11), 323.

Johnson, S. M. (1954). Optimal two-and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly, 1*, 61–68.

Joo, B.-J., & Kim, Y.-D. (2009). A branch-and-bound algorithm for a two-machine flowshop scheduling problem with limited waiting time constraints. *Journal of the Operational Research Society, 60*(4), 572–582.

Joo, B.-J., Kim, Y.-D., & Bang, J.-Y. (2009). A scheduling algorithm for workstations with limited waiting time constraints in a semiconductor wafer fabrication facility. *Journal of the Korean Institute of Industrial Engineers, 35*(4), 266–279.

Jung, C., Pabst, D., Ham, M., Stehli, M., & Rothe, M. (2014). An effective problem decomposition method for scheduling of diffusion processes based on mixed integer programming. *IEEE Transactions on Semiconductor Manufacturing, 27*(3), 357–363.

Kim, H.-J., & Lee, J.-H. (2019). Three-machine flow shop scheduling with overlapping waiting time constraints. *Computers & Operations Research, 101*, 93–102.

Kim, H.-I., & Lee, D.-H. (2023). Scheduling algorithms for multi-stage flow shops with reworks under overlapped queue time limits. *International Journal of Production Research, 61*(20), 6908–6922.

Klemmt, A., & Mönch, L. (2012). Scheduling jobs with time constraints between consecutive process steps in semiconductor manufacturing. In *Proceedings of the 2012 Winter Simulation Conference*, Berlin, Germany, pp. 2173–2182.

Komaki, G. M., Sheikh, S., & Malakooti, B. (2019). Flow shop scheduling problems with assembly operations: A review and new trends. *International Journal of Production Research, 57*(10), 2926–2955.

Lee, J.-Y. (2020). A genetic algorithm for a two-machine flowshop with a limited waiting time constraint and sequence-dependent setup times. *Mathematical Problems in Engineering, 2020*(1), Article 8833645.

Lee, J.-H. (2023). Scheduling of flow shop with sequence-dependent setup times and overlapping waiting time constraints to minimize the total completion time. *Journal of Korean Operations Research and Management Science Society, 48*(4), 39–51.

Lee, J.-H., Yu, T.-S., & Park, K. (2021). Scheduling of flow shop with overlapping waiting time constraints using genetic algorithm. *Journal of the Korean Institute of Industrial Engineers, 47*(1), 34–44.

Li, Y., & Dai, Z. (2020). A two-stage flow-shop scheduling problem with incompatible job families and limited waiting time. *Engineering Optimization, 52*(3), 484–506.

Mansouri, M., Bahmani, Y., & Smadi, H. (2023). Optimization of the flow-shop scheduling problem under time constraints with PSO algorithm. *Engineering Proceedings, 56*(1), 220.

Mladenović, N., & Hansen, P. (1997). Variable neighborhood search. *Computers & Operations Research, 24*(11), 1097–1100.

Mraihi, T., Driss, O. B., & El-Haouzi, H. B. (2024). Distributed permutation flow shop scheduling problem with worker flexibility: Review, trends and model proposition. *Expert Systems with Applications, 238*, Article 121947.

Ostermeier, F. F., & Deuse, J. (2024). A review and classification of scheduling objectives in unpaced flow shops for discrete manufacturing. *European Journal of Operational Research, 27*(1), 29–49.

Panwalkar, S. S., & Koulamas, C. (2020). Analysis of flow shop scheduling anomalies. *European Journal of Operational Research, 280*(1), 25–33.

Park, I.-B., & Huh, J. (2026). A genetic algorithm with feasibility-agnostic encoding and three-phase decoding for scheduling semiconductor manufacturing facilities under queue time limits. *Expert Systems with Applications, 301*, Article 130310.

Perez-Gonzalez, P., & Framinan, J. M. (2024). A review and classification on distributed permutation flowshop scheduling problems. *European Journal of Operational Research, 312*(1), 1–21.

Pirovano, G., Ciccullo, F., Pero, M., & Rossi, T. (2020). Scheduling batches with time constraints in wafer fabrication. *International Journal of Operational Research, 37*(1), 1–31.

Rossit, D. A., Tohmé, F., & Frutos, M. (2018). The non-permutation flow-shop scheduling problem: A literature review. *Omega, 77*, 143–153.

Rossit, D. A., Vásquez, Ó. C., Tohmé, F., Frutos, M., & Safe, M. D. (2021). A combinatorial analysis of the permutation and non-permutation flow shop scheduling problems. *European Journal of Operational Research, 289*(3), 841–854.

Su, L. H. (2003). A hybrid two-stage flowshop with limited waiting time constraints. *Computers & Industrial Engineering, 44*(3), 409–424.

Wang, H. K., Chien, C. F., & Gen, M. (2015). An algorithm of multi-subpopulation parameters with hybrid estimation of distribution for semiconductor scheduling with constrained waiting time. *IEEE Transactions on Semiconductor Manufacturing, 28*(3), 353–366.

Wang, B. L., Li, T. K., & Ge, H. B. (2013). Neighborhood search heuristic for 2-machine flowshop scheduling problem with limited waiting times. *Applied Mechanics and Materials, 411-414*, 1894–1897.

Wang, F., Su, Y., Ju, F., Ananthanarayanan, B., Dauod, H., & Patel, N. S. (2025). Fast schedule recovery method for semiconductor packaging lines with machine failures and constrained time windows. *IEEE Transactions on Automation Science and Engineering, 22*, 18076–18087.

Yang, D. L., & Chern, M. S. (1995). A two-machine flowshop sequencing problem with limited waiting time constraints. *Computers & Industrial Engineering, 28*(1), 63–70.

Yılmaz, B. G., & Yılmaz, Ö. F. (2022). Lot streaming in hybrid flowshop scheduling problem by considering equal and consistent sublots under machine capability and limited waiting time constraint. *Computers & Industrial Engineering, 173*, Article 108745.

Yılmaz, B. G., Yılmaz, Ö. F., & Yeni, F. B. (2024). Comparison of lot streaming division methodologies for multi-objective hybrid flowshop scheduling problem by considering limited waiting time. *Journal of Industrial and Management Optimization, 20*(11), 3373–3414.

Yu, T.-S., Kim, H.-J., & Lee, T.-E. (2017). Minimization of waiting time variation in a generalized two-machine flow shop with waiting time constraints and skipping jobs. *IEEE Transactions on Semiconductor Manufacturing, 30*(2), 155–165.

Zhou, N., Wu, M., & Zhou, J. (2018). Research on power battery formation production scheduling problem with limited waiting time constraints. In: *Proceedings of the International Conference on Communication Software and Networks*, Chengdu, China, pp. 497–501.